

Package ‘mizu’

September 30, 2026

Title Single-Producer Single-Consumer Channels

Version 0.0.1

Description Provides a single-producer single-consumer channel with bounded capacity, optional timeouts on receives, and sentinel return values for full, timeout, and closed states. The ring buffer is implemented in C: sends and receives are constant-time operations, values are delivered in first-in first-out order, and a closed channel drains its buffered values before reporting the closed state. The transport is in-process: both ends of a channel live in the calling R process, which suits the channel to prototyping producer and consumer designs and to testing channel-based logic.

License MIT + file LICENSE

URL <https://github.com/shikokuchuo/mizu-r>

BugReports <https://github.com/shikokuchuo/mizu-r/issues>

Depends R (>= 4.1)

Suggests testthat (>= 3.0.0)

Config/roxygen2/markdown TRUE

Config/roxygen2/version 8.1.0

Config/testthat/edition 3

Encoding UTF-8

NeedsCompilation yes

Author Charlie Gao [aut, cre] (ORCID: <<https://orcid.org/0000-0002-0750-061X>>),
Posit Software, PBC [cph, fnd] (ROR: <<https://ror.org/03wc8by49>>)

Maintainer Charlie Gao <charlie.gao@posit.co>

Repository CRAN

Date/Publication 2026-09-30 09:40:07 UTC

Contents

mizu_full	2
mizu_inproc_channel	2
mizu_inproc_close	3
mizu_is_sentinel	4
mizu_send	5
print.mizu_inproc_channel	6
Index	7

mizu_full	<i>Sentinel Values</i>
-----------	------------------------

Description

Singletons returned by `mizu_send()` and `mizu_recv()` that tag the terminal states of a channel as ordinary values, not signalled conditions, so hot loops stay branch-cheap. Dispatch with `inherits(x, "mizu_sentinel")` or on the specific class; use `mizu_is_sentinel()` where payloads are untrusted.

Usage

mizu_full

mizu_timeout

mizu_closed

Details

- mizu_full — the buffer is at capacity (`mizu_send()`).
- mizu_timeout — no value arrived within timeout (`mizu_recv()`).
- mizu_closed — the channel is closed and drained (`mizu_inproc_close()`).

mizu_inproc_channel	<i>Create a Channel</i>
---------------------	-------------------------

Description

Creates a single-producer single-consumer channel: a bounded ring buffer that holds up to capacity values until they are received. The ring buffer is implemented in C, with O(1) sends and receives. The transport is in-process — both ends of the channel live in the calling R process.

Usage

mizu_inproc_channel(capacity)

Arguments

capacity a positive integer: the number of values the channel buffers before `mizu_send()` returns the `mizu_full` sentinel.

Details

This interface is experimental and may change in a future release.

Value

A channel (class "mizu_inproc_channel").

Examples

```
ch <- mizu_inproc_channel(2)
mizu_send(ch, "a")
mizu_send(ch, 42)
mizu_recv(ch)
mizu_recv(ch)
mizu_inproc_close(ch)
```

mizu_inproc_close	<i>Close a Channel</i>
-------------------	------------------------

Description

Orderly shutdown. Values buffered before the close remain receivable; further sends return the `mizu_closed` sentinel, and a receive on a drained channel returns the same. Closing an already-closed channel is a no-op.

Usage

```
mizu_inproc_close(ch)
```

Arguments

ch a channel from `mizu_inproc_channel()`.

Value

Invisibly, TRUE.

Examples

```
ch <- mizu_inproc_channel(1)
mizu_send(ch, "a")
mizu_inproc_close(ch)
mizu_recv(ch)
mizu_recv(ch)
```

mizu_is_sentinel	<i>Test for a mizu Sentinel</i>
------------------	---------------------------------

Description

Identity comparison against the interned sentinel singletons — [mizu_full](#), [mizu_timeout](#), and [mizu_closed](#) — that the verbs of mizu return to tag terminal states. `inherits(x, "mizu_sentinel")` tests the class alone, which any payload can carry. This includes a genuine sentinel forwarded over a channel, which arrives as an ordinary copy. `mizu_is_sentinel()` is provenance: TRUE only for the exact objects that the own calls of mizu return. So code that relays untrusted values can distinguish its terminal states from look-alike payloads.

Usage

```
mizu_is_sentinel(x)
```

Arguments

`x` any R object.

Details

Sentinels are ordinary values, not R conditions: nothing is signalled, and condition handlers never see them.

Value

TRUE or FALSE.

Examples

```
ch <- mizu_inproc_channel(1)
x <- mizu_recv(ch, timeout = 0)
mizu_is_sentinel(x)
mizu_is_sentinel(42)
# class alone does not make a sentinel:
mizu_is_sentinel(
  structure("x", class = c("mizu_timeout", "mizu_sentinel"))
)
mizu_inproc_close(ch)
```

mizu_send

Send and Receive over a Channel

Description

mizu_send() appends `x` to the buffer of the channel. Sends never block: a send on a full channel returns the `mizu_full` sentinel immediately. `mizu_recv()` removes and returns the oldest buffered value, waiting up to `timeout` seconds for one to arrive.

Usage

```
mizu_send(ch, x)
```

```
mizu_recv(ch, timeout = Inf)
```

Arguments

<code>ch</code>	a channel from <code>mizu_inproc_channel()</code> .
<code>x</code>	the payload: any R object.
<code>timeout</code>	seconds to wait before the call returns the <code>mizu_timeout</code> sentinel. <code>Inf</code> (the default) waits indefinitely, and <code>0</code> polls. The wait is a bounded spin-then-backoff poll on a monotonic clock: system clock adjustments do not affect it, and Ctrl-C stays responsive throughout.

Details

Terminal states surface as class-tagged sentinels, not errors. Dispatch with `inherits(x, "mizu_sentinel")`, or on the specific classes:

- `mizu_full` — the buffer is at capacity (send). Receive first, or drop the value.
- `mizu_timeout` — no value arrived within `timeout` (recv).
- `mizu_closed` — the channel was closed with `mizu_inproc_close()`. A receive drains all buffered values before it reports this.

NULL is a legal payload. Sentinels are ordinary values, identifiable by class alone, and never signalled conditions. `mizu_is_sentinel()` checks identity where payloads are untrusted.

Value

`mizu_send()` returns TRUE (invisibly) on success, or a sentinel otherwise. `mizu_recv()` returns the received value or a sentinel.

Examples

```
ch <- mizu_inproc_channel(1)
mizu_send(ch, "a")
mizu_send(ch, "b") # full: returns the mizu_full sentinel
mizu_recv(ch)
mizu_recv(ch, timeout = 0)
mizu_inproc_close(ch)
```

```
print.mizu_inproc_channel
```

Print Methods for mizu Objects

Description

One-line summaries. A channel prints its state — open or closed — and the number of buffered values out of its capacity. Sentinels print as their class. The methods never error, so auto-printing is always safe.

Usage

```
## S3 method for class 'mizu_inproc_channel'
print(x, ...)

## S3 method for class 'mizu_sentinel'
print(x, ...)
```

Arguments

x	the object.
...	ignored.

Value

x, invisibly.

Examples

```
ch <- mizu_inproc_channel(2)
ch
mizu_send(ch, "a")
ch
mizu_inproc_close(ch)
```

Index

mizu_closed, [4](#)
mizu_closed (mizu_full), [2](#)
mizu_full, [2](#), [3](#), [4](#)
mizu_inproc_channel, [2](#)
mizu_inproc_channel(), [3](#), [5](#)
mizu_inproc_close, [3](#)
mizu_inproc_close(), [2](#), [5](#)
mizu_is_sentinel, [4](#)
mizu_is_sentinel(), [2](#), [5](#)
mizu_recv (mizu_send), [5](#)
mizu_recv(), [2](#)
mizu_send, [5](#)
mizu_send(), [2](#), [3](#)
mizu_timeout, [4](#)
mizu_timeout (mizu_full), [2](#)

print.mizu_inproc_channel, [6](#)
print.mizu_sentinel
 (print.mizu_inproc_channel), [6](#)